Lecture–4: Data Input & Output in Pandas DataFrame

Topics Covered

1. Data Input & Output from CSV
2. Data Input & Output from Excel
3. Data Input & Output from SQL
4. Data Input & Output from JSON
5. View Data using head() and tail()
6. Shape of DataFrame
7. Select Single Column
8. Select Multiple Columns

---

1. Why Data Input & Output is Important?

Real companies store data in:

- CSV files
- Excel files
- SQL databases
- JSON files
- APIs

A Data Analyst must know how to:

Read data → Analyze → Clean → Save output

Real-world Example

Suppose Amazon provides:

customer.csv

You must:

Read file

↓

Analyze purchase data

↓

Generate report

↓

Export cleaned dataset

Pandas Input & Output Functions

File Type	Read Function	Write Function
CSV	read_csv()	to_csv()
Excel	read_excel()	to_excel()
SQL	read_sql()	to_sql()
JSON	read_json()	to_json()

Remember:

read → input

to_ → output

---

2. Data Input & Output using CSV

What is CSV?

CSV:

Comma Separated Values

Example:

Name,Age,Salary

Ram,22,40000

Rita,25,50000

Looks like Excel but stored as text.

Read CSV File

Syntax:

pd.read_csv("filename.csv")

Example:

Suppose:

students.csv

Contains:

Name	Age
Ram	22
Rita	25

Read:

```
import pandas as pd

df=pd.read_csv("students.csv")

print(df)
```

Output:

Name	Age
Ram	22
Rita	25

Save DataFrame to CSV

Syntax:

df.to_csv("newfile.csv")

Example:

df.to_csv("output.csv")

Creates:

output.csv

Remove Index while Saving

Default:

Pandas saves row numbers.

Avoid:

```
df.to_csv(
"output.csv",
index=False
)
```

Recommended.

Important CSV Parameters

Read only first 5 rows

```
pd.read_csv(
"data.csv",
nrows=5
)
```

Select specific columns

```
pd.read_csv(
"data.csv",
usecols=["Name"]
)
```

Change separator

If data separated by ";"

```
pd.read_csv(
"data.csv",
sep=";"
)
```

Interview Question

Difference:

read_csv()

vs

to_csv()

Answer:

read = input

to_csv = output

---

3. Data Input & Output using Excel

Need package:

Install:

pip install openpyxl

Read Excel File

Syntax:

pd.read_excel()

Example:

employees.xlsx

Read:

```
df=pd.read_excel(
"employees.xlsx"
)

print(df)
```

Output:

Name	Salary
Ram	40000

Read Specific Sheet

Suppose Excel has:

Sheet1

Sheet2

Read:

```
pd.read_excel(
"file.xlsx",

sheet_name="Sheet2"
)
```

Write DataFrame to Excel

Save:

```
df.to_excel(
"output.xlsx"
)
```

Remove index:

```
df.to_excel(
"output.xlsx",

index=False
)
```

Example

Create DataFrame:

```
student={

"Name":["Ram","Rita"],

"Age":[22,25]
```

```
}

df=pd.DataFrame(student)

df.to_excel(
"student.xlsx",

index=False
)
```

Excel file created.

---

4. Data Input & Output using SQL

Companies mostly use:

- MySQL
- PostgreSQL
- SQL Server

Analysts often read SQL data into Pandas.

Install SQLAlchemy

```
pip install sqlalchemy
```

Read SQL Table

Syntax:

```
pd.read_sql()
```

Example:

```
import sqlite3

conn=sqlite3.connect(
"company.db"
)

df=pd.read_sql(
```

"SELECT * FROM employee",

conn

)

print(df)

Output:

Name Salary

Ram 50000

Save DataFrame to SQL

df.to_sql(

"employee",

conn

)

Example

df.to_sql(

"student",

conn,

if_exists="replace"

)

Parameter:

replace

append

fail

Real-world Flow

SQL Database

↓

Pandas

↓

Analysis

↓

Dashboard

---

5. Data Input & Output using JSON

What is JSON?

JSON:

JavaScript Object Notation

Example:

```
[
{
"Name":"Ram",

"Age":22
}
]
```

Used heavily in APIs.

Read JSON

Syntax:

pd.read_json()

Example:

```
df=pd.read_json(
"student.json"
)

print(df)
```

Output:

Name	Age
Ram	22

Save JSON

```
df.to_json(
"output.json"
)
```

Example

```
student={

"Name":["Ram"],

"Age":[22]

}

df=pd.DataFrame(student)

df.to_json(
"student.json"
)
```

---

6. View Data in DataFrame

Often datasets have:

10 lakh rows

Cannot print all.

Need preview.

---

head()

Shows first rows.

Default:

5 rows

Syntax:

df.head()

Example:

print(df.head())

Output:

First 5 rows.

Custom rows

Show first 10:

df.head(10)

---

tail()

Shows last rows.

Default:

5 rows

Syntax:

df.tail()

Example:

df.tail(2)

Shows last 2 rows.

Why Important?

Interview Question:

Why use head()?

Answer:

Preview dataset quickly.

Example

Dataset:

Name	Age
A	20
B	21
C	22

df.head()

Shows top rows.

---

df.tail()

Shows bottom rows.

---

7. Shape of DataFrame

Need to know:

Rows?

Columns?

Use:

df.shape

Example:

Dataset:

Name	Age
A	20
B	25

Output:

(2,2)

Meaning:

2 rows

2 columns

Interpretation:

(rows,columns)

Remember.

Example

print(df.shape)

Output:

(1000,5)

Meaning:

1000 rows

5 columns

Interview Question

What does:

(10,3)

mean?

Answer:

10 rows

3 columns

---

8. Select Single Column

Suppose:

Name	Age
Ram	22

Method 1

Syntax:

df["column"]

Example:

df["Name"]

Output:

Ram

Rita

Series returned.

Method 2

Dot notation

df.Name

Not always recommended.

Recommended:

df["Name"]

Example

```
student={

"Name":["Ram","Rita"],

"Age":[22,25]

}

df=pd.DataFrame(student)

print(df["Age"])
```

Output:

22

25

---

9. Select Multiple Columns

Need:

Name + Age

Syntax:

Double brackets

```
df[
["Name","Age"]
]
```

Example:

```
print(

df[

["Name","Age"]

]
```

)

Output:

Name	Age
Ram	22

Important:

Single column:

df["Name"]

One bracket inside

Multiple:

df[["Name","Age"]]

Double brackets

Why Double Brackets?

Because:

["Name","Age"]

is a Python list.

Pandas receives list of columns.

Example Dataset

employee={

"Name":["A","B"],

"Age":[20,21],

"Salary":[30000,40000]

}

df=pd.DataFrame(employee)

Select:

Only Salary:

df["Salary"]

Name + Salary:

df[
["Name","Salary"]
]

Output:

Name	Salary
A	30000

Summary Table

Operation	Syntax
Read CSV	read_csv()
Save CSV	to_csv()
Read Excel	read_excel()
Save Excel	to_excel()
Read SQL	read_sql()
Save SQL	to_sql()
Read JSON	read_json()
Save JSON	to_json()
Top rows	head()
Last rows	tail()

Operation	Syntax
Shape	shape
Single column	df["col"]
Multiple columns	df[["c1","c2"]]

Common Interview Questions

Q1 Difference:

head()

and

tail()

Answer:

head → top rows

tail → bottom rows

Q2 Meaning of:

(100,5)

Answer:

100 rows

5 columns

Q3 Why use double brackets?

Answer:

To select multiple columns.

Q4 Which function reads CSV?

pd.read_csv()

Lecture–4 Summary

Students should know:

✓ Read CSV

✓ Read Excel

✓ Read SQL

✓ Read JSON

✓ Save datasets

✓ Use head()

✓ Use tail()

✓ Find shape

✓ Select columns

✓ Select multiple columns

Homework Practice

Create DataFrame:

student={

"Name":["Ram","Rita","Aman"],

"Age":[22,25,24],

"Salary":[40000,50000,45000]

}

Questions:

1. Show first 2 rows
2. Show last row
3. Find shape
4. Select Age column
5. Select Name and Salary columns
6. Save dataset as CSV
7. Save dataset as Excel